CSC3002 | Midterm revision guide

Lectures 1-9 • Definitions, rules and worked examples

Scope: the nine lecture slide decks saved in your course folder, following your confirmed midterm coverage. Technical topics are consolidated; repeated slides and course administration are omitted. This is a study guide, not an official exam prediction.

Each topic gives the definition, the rule to remember, and a small example. Code uses C++17, ordinary ASCII examples, and std:: qualification unless a snippet explicitly uses a using declaration. Snippets are independent; include the headers listed for each topic. Expressions shown alone are illustrative, not complete programs.

Navigation

Lecture	Topic	Pages
1	C++ foundations; expressions and control flow	2-3
2	Functions; arguments, references and operators	4-5
3	Libraries, interfaces and pseudorandom numbers	6-7
4	Characters, C strings and C++ strings	8-9
5	Streams, formatting, files and string streams	10-11
6	Memory, pointers, arrays and pointer arithmetic	12-14
7	Dynamic allocation, lifetime, const/static and smart pointers	15-16
8	Classes, copying, operators, inheritance and polymorphism	17-20
9	Templates, generic sorting and iterators	21-22
Review	Tracing checklist and source map	23-24

How to use this guide

First learn each bold term. Then cover the comments in the examples and predict the result. For pointers, draw separate boxes for the pointer and the object it points to. For classes, track construction, copying, mutation and destruction separately.

Three possible exam answers: a valid program has a traceable result; a compile error prevents translation; undefined behavior means C++ guarantees no particular result. Never invent an output for undefined behavior.

Linked lists: not taught as a standalone algorithm topic in these nine decks. Pointer and allocation knowledge is included; linked-list insertion/deletion is outside this guide. The stack appears as a class-design example, and selection sort appears as a template example, so both are included.

Accuracy note: some slide examples contain typos, nonstandard signatures, or unsafe operations. Examples here use valid syntax and explicitly explain the intended behavior. When an exam supplies a function body, trace that body even if it uses an unusual operator convention.

Lecture 1 | C++ foundations

1.1 Program structure and translation

Source file: human-readable C++ code, usually .cpp. **Header:** declarations and type definitions shared by source files. **Preprocessor:** processes directives such as #include and #define before compilation. **Compiler:** translates a translation unit into object code. **Object file:** compiled code that may contain unresolved external names. **Linker:** combines object files and libraries into an executable. **Executable:** program the operating system can run.

```
#include <iostream>             // Include declarations
int main() {                    // Program entry point
    std::cout << "Hello\n";
    return 0;                   // Successful termination
}
```

Declaration: introduces a name and its type. **Definition:** supplies the entity itself, such as a variable's storage or a function's body. **Comment:** ignored explanatory text: // to end of line, or /* ... */ across lines.

```
g++ -c math.cpp -o math.o        # Compile without linking
ar rcs libmath.a math.o          # Archive an object library
g++ main.cpp libmath.a -o app    # Compile and link
```

A **build script** runs build commands; a **Makefile** uses targets and dependencies to describe a build. A shell script is not itself Makefile syntax.

1.2 Names, types, variables and constants

Namespace: scope that groups names to avoid collisions. **Library:** reusable code exposed through an interface. std is the standard-library namespace; std::cout selects cout in that namespace. using namespace std; makes names available to unqualified lookup; avoid it in public headers.

Type: specifies a set of values and permitted operations. **Primitive types:** int for integers, double for floating-point approximations, char for character code units, bool for true/false. short/long and unsigned variants provide other integer ranges; float/long double provide other floating-point types.

Variable: named object with a type, value, storage and lifetime. **Identifier:** case-sensitive name; use letters, digits and underscores, not a leading digit or keyword. **Initialization:** supplies an initial value. **Assignment:** replaces an existing value.

```
int total = 42;       // Definition and initialization
total = 9;            // Assignment
const double PI = 3.141592653589793;
// PI = 3;            // Compile error: const object
```

Literal: value written directly: 42, 3.5, 5.0e-3, 'a', "hi", true. **Named constant:** const object whose value cannot be modified through its name. #define performs token substitution; it does not create a typed variable.

1.3 Scope and lifetime

Scope: where a name is visible. **Lifetime (extent):** when an object exists. **Local variable:** declared inside a block; an ordinary automatic local lives until its block exits. **Global variable:** declared outside functions/classes. **Member/instance variable:** belongs to an object. A local can hide an outer name (**shadowing**).

```
int x = 1;                    // Global
void f() {
    int x = 2;                // Hides global x
    { int y = x; }            // y's scope/lifetime end here
}                             // Local x ends here
```

Pitfall: int x; does not initialize an automatic int. Initialize before reading it. Exact object sizes and addresses depend on the implementation.

Lecture 1 | Expressions and control flow

1.4 Expressions and operators

Expression: combination of values/names/operators/calls that performs a computation. **Operand:** value an operator acts on. **Unary/binary/ternary operator:** takes one/two/three operands. Arithmetic: + - * / %. Assignment: = += -= *= /= %=.

Integer division truncates toward zero. % gives the integer remainder, with a = (a/b)*b + a%b when division is defined. Division by zero is undefined behavior for integers.

```
14 / 5;                    // 2
static_cast<double>(14 / 5); // 2.0: conversion is too late
14 / 5.0;                  // 2.8
-14 / 5;                   // -2
-14 % 5;                   // -4
```

Type conversion/cast: changes the type used to represent/evaluate a value. Mixed int/double arithmetic generally converts the int to double. Floating-to-integer conversion truncates toward zero when representable.

Precedence: how operators bind. **Associativity:** how equal-precedence operators group; neither generally specifies operand evaluation order. Useful high-to-low order: postfix () [] . -> and postfix ++/--; unary ! * & ++/--; * / %; + -; relational < <= > >=; equality == !=; &&; ||; ?:; assignment. Use parentheses when unsure.

```
int a = 5;
int b = a++;     // b=5, a=6: postfix yields old value
a = 5;
int c = ++a;     // c=6, a=6: prefix yields updated value
```

Boolean expression: produces true/false. Relational operators compare; ! negates; && means AND; || means OR. Zero converts to false; nonzero to true. **Short-circuit evaluation:** built-in && skips its right side when left is false; || skips it when left is true.

```
bool safe = (n != 0 && 10 / n > 2); // Division skipped if n=0
bool inRange = (0 < x && x < 10);   // Correct chained condition
// 0 < x < 10 means (0 < x) < 10; legal, usually wrong.
```

1.5 Statements and control flow

Statement: instruction executed by a program. **Block/compound statement:** statements enclosed in braces. **Selection:** choose a branch with if/else or switch. **Conditional expression:** condition ? valueIfTrue : valueIfFalse.

```
int magnitude = (x < 0) ? -x : x; // Assume -x is representable
switch (choice) {
    case 1: std::cout << "one"; break;
    default: std::cout << "other";
}
```

switch: selects integral/enum cases. **break:** exits the nearest loop or switch; omitting it permits fall-through. **Loop/iteration:** repeated execution. while tests before each iteration. for runs initialization once, then test, body, step, repeat. **continue:** skips the remaining body; in a for loop it still runs the step.

```
int sum = 0;
for (int i = 1; i <= 3; ++i) sum += i; // sum=6
int i = 3;
while (i > 0) --i;                     // i=0
```

Unit test: checks a small program component against expected behavior. **Assertion:** runtime check of an assumed condition; #include <cassert> and assert(2 + 2 == 4);. Assertions can be disabled with NDEBUG; do not use them for essential work.

Pitfalls: = assigns while == compares; a stray semicolon after if/for/while creates an empty statement; off-by-one errors include/exclude the wrong endpoint.

Lecture 2 | Functions and call mechanics

2.1 Functions, parameters and return values

Function: named reusable unit of computation. **Call:** invokes a function. **Parameter:** variable declared in a function's parameter list. **Argument:** expression supplied at the call. **Return type:** type of result. **Body:** statements implementing the function. **Predicate:** function returning bool. **Procedure:** function returning void (no result value).

```
int square(int x);                  // Prototype/declaration
int square(int x) { return x * x; }  // Definition
bool isEven(int n) { return n % 2 == 0; }
void printSquare(int n) { std::cout << square(n); }
// square(4) returns 16: x is a parameter, 4 is an argument.
```

Prototype: function declaration ending with a semicolon; lets the compiler check calls before the definition appears. A definition placed before a call also supplies the declaration. return expression; supplies the result and exits immediately; return; exits a void function.

Abstraction: using what an operation does without managing its internal steps. **Decomposition:** breaking a larger task into smaller functions. **Algorithm:** finite, well-specified procedure for solving a problem.

2.2 Pass by value and stack frames

Pass by value: initializes a separate parameter object from the argument. Changes to that parameter do not change the caller's object. **Stack frame:** conceptual per-call storage for parameters, local variables and return information. Each call has its own locals; execution resumes after the call when it returns.

```
void change(int n) { n = 99; }
int x = 5;
change(x);                          // x remains 5
```

Argument positions match parameter positions. Do not assume arguments are evaluated from left to right; avoid examples that depend on competing side effects in argument expressions.

2.3 Factorial and combinations

Factorial: n! = 1*2*...*n, with 0! = 1. **Combination:** number of ways to choose k items from n without order: C(n,k) = n! / (k! * (n-k)!). These illustrate function reuse; intermediate factorials can overflow even when an answer would fit.

```
int fact(int n) {                   // Precondition: 0 <= n <= 12
    int result = 1;
    for (int i = 2; i <= n; ++i) result *= i;
    return result;
}
int choose(int n, int k) {          // Small nonnegative inputs
    return fact(n) / (fact(k) * fact(n - k));
}
// choose(5, 2) == 10; require 0 <= k <= n.
```

2.4 Greatest common divisor

GCD: greatest positive integer dividing both inputs. **Brute force:** tests candidate solutions individually. **Euclid's algorithm:** repeatedly applies gcd(a,b) = gcd(b,a%b), stopping when b=0. For positive inputs, it avoids scanning all smaller numbers.

```
int gcd(int a, int b) {             // a,b positive
    while (b != 0) {
        int r = a % b;
        a = b;
        b = r;
    }
    return a;
}
// (78,33) -> (33,12) -> (12,9) -> (9,3) -> (3,0)
// gcd(78,33) == 3
```

Precondition: requirement before a call. **Postcondition:** promised result afterward. Example: positive inputs before gcd; a common divisor afterward.

Lecture 2 | References, overloads and operators

2.5 Reference variables and reference parameters

Reference: alias for an existing object; must be initialized and cannot be reseated to another object. **Pass by reference:** parameter aliases the caller's object, allowing modification and avoiding an object copy. **Const reference:** read-only access through an alias, commonly used for large inputs.

```
int a = 1, b = 2;
int& r = a;
r = b;                  // Assigns 2 to a; r still aliases a
void swapInts(int& x, int& y) {
    int t = x; x = y; y = t;
}
// swapInts(a,b) exchanges the caller's values.
std::size_t lengthOf(const std::string& s) { return s.size(); }
```

An ordinary non-const int& parameter needs a suitable modifiable lvalue: swapInts(1,2) is invalid. A const reference can bind to a temporary. Returning a reference to a local automatic object creates a dangling reference.

2.6 Overloading and default arguments

Function overloading: multiple functions share a name but differ in parameter lists. **Signature (for these ordinary functions):** function name and parameter types/order; parameter names and return type alone do not distinguish overloads. **Overload resolution:** compiler selects the best viable match using argument types.

```
int half(int n) { return n / 2; }
double half(double n) { return n / 2; }
// half(3) == 1; half(3.0) == 1.5
// double half(int n); cannot overload int half(int n).
```

Default argument: value used when a trailing argument is omitted. Put defaults once in the declaration visible to callers. After a parameter has a default, subsequent ordinary parameters must have defaults too.

```
void locate(double x = 0, double y = 0);
// locate() -> (0,0); locate(3) -> (3,0); locate(3,4) -> (3,4)
// Cannot omit x but supply only y using positional arguments.
```

To permit only zero or two arguments, provide locate() and locate(double,double) as separate overloads, without defaults on the two-argument version.

2.7 Operator overloading

Operator overload: user-defined function giving an existing operator meaning for a class/enum type. It cannot change precedence, associativity or operand count, and cannot redefine an operator solely for primitive operands.

```
struct Point { int x, y; };
Point operator+(const Point& a, const Point& b) {
    return {a.x + b.x, a.y + b.y};
}
Point& operator++(Point& p) {       // Prefix: ++p
    ++p.x; ++p.y; return p;
}
Point operator++(Point& p, int) {   // Postfix: p++
    Point old = p; ++p; return old;
}
```

The dummy int distinguishes postfix from prefix; it is not an extra value you supply in p++. Conventional + returns a new value; += mutates the left operand. An exam's supplied operator may behave differently: read the implementation.

Free function: function outside a class. A member binary operator has the left operand as *this and one explicit operand. A free binary operator has two explicit operands. Assignment operator= must be a non-static member in C++17.

Lecture 3 | Libraries and interfaces

3.1 Client, interface and implementation

Library: reusable types and operations. **Client:** code using a library. **Implementer:** code/author providing its behavior. **Interface:** public contract of names, types and behavior. **Implementation:** hidden code fulfilling that contract. **Information hiding:** keeping changeable representation details away from clients.

```
// mathutil.h
#ifndef MATHUTIL_H
#define MATHUTIL_H
int add(int a, int b);
extern const double PI;
#endif

// mathutil.cpp
#include "mathutil.h"
int add(int a, int b) { return a + b; }
const double PI = 3.141592653589793;
```

Include guard: #ifndef/#define/#endif prevents repeated processing of a header within one translation unit; #pragma once is a widely supported alternative. Include needed headers in the header itself. Guards do not fix multiple non-inline function definitions across different translation units.

External linkage: a name can refer to the same entity across translation units. **Internal linkage:** name denotes an entity local to one translation unit. A namespace-scope non-extern const usually has internal linkage. extern const double PI; declares a shared constant; define it once. The included extern declaration above gives its subsequent definition external linkage.

#include <...>: conventional library-header lookup. **#include "...":** conventional local-header lookup first; actual search paths depend on the compiler. Header suffixes alone do not determine semantics.

3.2 Standard-library categories and math functions

Standard library: facilities specified with C++, mostly in std. Headers: <iostream> console I/O; <string> strings; <cmath> math; <cctype> character tests; <cstring> C-string functions; <cstdlib> C utilities; <fstream> files; <sstream> string streams; <memory> smart pointers; <vector>/<algorithm>/<iterator> collections and algorithms.

Function in <cmath>	Meaning	Example
std::abs(x)	Absolute value	abs(-3) = 3
std::sqrt(x)	Square root	sqrt(9.0) = 3.0
std::floor(x)	Greatest integer value <= x	floor(-2.3) = -3.0
std::ceil(x)	Smallest integer value >= x	ceil(-2.3) = -2.0
std::exp(x), std::log(x)	e^x, natural logarithm	log(1.0) = 0.0
std::log10(x), std::pow(x,y)	Base-10 log, x^y	pow(2.0,3) = 8.0
std::sin/cos/tan(x)	Trigonometric functions in radians	sin(0.0) = 0.0
std::atan(x), std::atan2(y,x)	Inverse tangent; quadrant-aware angle	atan2(1.0,0.0) is pi/2

3.3 Five interface-design principles

Unified: one coherent purpose. **Simple:** easy for clients to use. **Sufficient:** supplies the needed operations. **General:** useful across clients. **Stable:** preserves the public contract as internals change.

Example: a random-number library should offer randomInteger(low,high), rather than forcing every client to repeat scaling arithmetic. Changing its internal generator should preserve the promised interval.

Lecture 3 | Enumerations and randomness

3.4 Exporting an enumeration

Enumeration (enum): distinct type with named integer constants. Unspecified values start at 0 and increase by 1. A cast can convert an integer result back to an enum; keep results within the intended range.

```
enum Direction { NORTH, EAST, SOUTH, WEST };
Direction rightFrom(Direction d) {
    return Direction((d + 1) % 4);
}
Direction leftFrom(Direction d) {
    return Direction((d + 3) % 4);
}
// rightFrom(WEST) == NORTH; leftFrom(NORTH) == WEST
```

Adding 3 avoids a negative remainder from (d-1)%4 when d=NORTH. Explicit values are allowed: enum Coin { PENNY=1, NICKEL=5, DIME=10 };.

Stream insertion overload: formats a new type and returns the same stream by reference so output can be chained.

```
std::ostream& operator<<(std::ostream& os, Direction d) {
    const char* names[] = {"NORTH", "EAST", "SOUTH", "WEST"};
    return os << names[static_cast<int>(d)]; // Valid d assumed
}
```

3.5 Pseudorandom numbers and seeding

Deterministic: same starting state and inputs give the same results. **Pseudorandom sequence:** generated deterministically but intended to appear random. **Seed:** initializes generator state; same seed reproduces the same sequence for the same implementation.

std::rand() in <cstdlib> returns an int in [0,RAND_MAX]. std::srand(seed) resets its state. Seeding once from time can vary runs; reseeding before every draw can repeat values. rand is not a source of cryptographic randomness.

Closed interval [a,b]: includes both endpoints. **Half-open interval [a,b):** includes a, excludes b. The course library promises inclusive random integers and half-open random reals.

```
// Course-style scaling; assume valid modest finite bounds.
double d = std::rand() / (double(RAND_MAX) + 1.0);
// Mathematically 0 <= d < 1.
int roll = int(std::floor(1 + d * 6)); // Intended integers 1..6
double real = low + d * (high - low); // Intended [low,high)
bool event = (d < p);                 // Probability p, 0 <= p <= 1
```

The casts prevent integer division; converting before RAND_MAX+1 avoids signed integer overflow. Finite generator resolution and floating-point rounding limit ideal distribution/endpoint claims. The course formulas are not a guarantee of perfectly uniform arbitrary integer ranges.

randomChance(p): true with the specified probability; randomBool() uses p=0.5. **setRandomSeed(seed):** explicitly chooses reproducible state for debugging. These are course-library functions, not standard C++ functions.

```
void initSeed() {
    static bool initialized = false;
    if (!initialized) {
        std::srand(static_cast<unsigned>(std::time(nullptr)));
        initialized = true;
    }
}
```

Local static: initialized once when control first reaches its declaration, retains state across calls, and lasts until program termination. Required headers here: <cstdlib>, <cmath>, <ctime>.

Lecture 4 | Characters and C strings

4.1 Characters and character classification

Character: one code unit represented by char. sizeof(char) is always 1 C++ byte; a byte is typically 8 bits. **ASCII:** 7-bit encoding with 128 codes; typical course examples use ASCII. **Unicode:** character standard covering many writing systems; a plain char is not generally a whole Unicode character.

Character literal: 'a'. **String literal:** "a", an array of two const char elements: 'a' and '\0'. **Escape sequence:** special spelling inside literals: '\n' newline, '\t' tab, '\r' carriage return, '\0' null character, '\\' backslash, '\'' single quote, '\"' double quote.

```
char digit = '7';
int value = digit - '0';   // 7, not the code value of '7'
// Under ASCII: 'A'=65, 'a'=97, '0'=48, space=32.
```

<cctype> functions: isdigit tests digits; isalpha letters; isalnum letters/digits; islower/isupper case; isspace whitespace (also newline, tab, etc.); tolower/toupper convert case if possible. Classification returns int interpreted as zero/nonzero, not necessarily bool or exactly 1.

```
char ch = 'a';
bool letter = std::isalpha(static_cast<unsigned char>(ch)) != 0;
char upper = static_cast<char>(
    std::toupper(static_cast<unsigned char>(ch))); // 'A'
```

Pass EOF or a value representable as unsigned char to <cctype>; casting a possibly negative char prevents invalid calls. Results for letters can depend on locale; examples assume ordinary ASCII letters.

4.2 C strings and their storage

C string: character sequence terminated by '\0'. **Buffer:** array storing data. **Capacity:** space available; **length:** number of characters before the terminator. A buffer needs at least length+1 elements.

```
char s[] = "hello";          // 6 elements, including '\0'
std::strlen(s);              // 5: <cstring>
sizeof s;                    // 6: complete array in this scope
char raw[] = {'h','i'};      // Not a C string: no terminator
const char* literal = "hi";  // Do not modify the literal
```

<cstring> operation	Definition / rule	Example
strlen(s)	Counts characters before '\0'	strlen("hi") == 2
strcpy(dst,src)	Copies all characters and '\0'	char b[8]; strcpy(b,"hi");
strcat(dst,src)	Appends src, including final '\0'	strcat(b,"!"); gives "hi!"
strcmp(a,b)	Lexicographic comparison; negative/zero/positive	strcmp("a","b") < 0

strchr(s,ch) finds a character and strstr(s,text) finds a substring; they return a pointer to the match or nullptr. These functions require valid terminated input. Copy/append need enough destination space and nonoverlapping regions.

Pitfalls: arrays cannot be reassigned: s="world" is invalid. strcpy(s,"hello world") overflows the 6-element buffer: undefined behavior. Comparing char* values with == compares pointer values, not text.

Lecture 4 | C++ strings and applications

4.3 std::string as an abstract data type

Abstract data type (ADT): defined by its operations/behavior rather than internal representation. **Class:** user-defined type bundling state and operations. **Object/instance:** value of that type. **Method:** member function called as object.method(...). std::string in <string> manages a variable-length character sequence.

```
std::string s = "hello";
std::string copy = s;        // Independent value
copy[0] = 'H';              // s remains "hello"
s += "!";                   // s is "hello!"
```

Concatenation: joining sequences end to end. At least one operand of string + must be a std::string (or suitable string overload operand); two C string literals cannot be added.

```
std::string good = std::string("Hello, ") + "world";
// "Hello, " + "world" is a compile error.
```

Lexicographic comparison: compare at the first differing character; if one sequence is a prefix, the shorter is smaller. std::string supports == != < <= > >= on contents. Case matters; ASCII "Z" < "a".

4.4 Access, search and modification

Operation	Definition	Example, independently using s="banana"
s.size(), s.length()	Number of stored characters	6
s[i]	Element access, no bounds exception	s[1] is 'a'
s.at(i)	Checked element access	s.at(6) throws out_of_range
s.substr(pos,count)	New substring; does not modify s	s.substr(1,3) is "ana"
s.find(text,pos)	First match at/after pos, or npos	s.find("na",3) is 4
s.erase(pos,count)	Removes characters from s	s.erase(1,2) gives "bana"
s.insert(pos,text)	Inserts before pos	s.insert(0,"X") gives "Xbanana"
s.replace(pos,count,text)	Replaces a segment	s.replace(0,3,"X") gives "Xana"
s.c_str()	Pointer to null-terminated view	strlen(s.c_str()) is 6 here

Use std::string::npos for failed find; its type is an unsigned size type, not an ordinary negative index. substr/erase counts can extend past the end and are limited by remaining characters; a starting position greater than size() throws. Modifiable character indices are 0..size()-1. Never write a non-null character at s[size()]. A c_str() pointer can be invalidated by string mutation/destruction.

Mutable: contents can change. **Immutable:** contents cannot change. C++ strings are mutable. A string passed by value is copied; modifying that copy preserves the caller's string. std::string& allows in-place mutation; const std::string& gives read-only access without copying.

4.5 Traversal, reversing, palindromes and acronyms

Traversal: visit elements in order. **Palindrome:** reads the same forward/backward. **Acronym:** initial letters of words. **Reverse:** sequence with opposite element order.

```
bool isPalindrome(const std::string& s) {
    for (std::size_t i = 0; i < s.size()/2; ++i)
        if (s[i] != s[s.size()-1-i]) return false;
    return true;
}
std::string reversed(const std::string& s) {
    std::string r;
    for (std::size_t i = s.size(); i > 0; --i) r += s[i-1];
    return r;                   // Handles an empty string safely
}
```

For acronyms, take a character when it starts a word: non-whitespace now and either index 0 or whitespace before it. Define what counts as a word first (e.g., whether hyphens split words).

Lecture 5 | Streams and formatted I/O

5.1 Stream definitions and operators

Stream: object representing a flow of characters to/from a source or destination. **Input stream:** reads; **output stream:** writes. **Formatted I/O:** converts between text and typed values. **Unformatted I/O:** reads/writes characters without numeric parsing. Standard <iostream> objects: std::cin standard input; std::cout standard output; std::cerr standard error.

Insertion <<: sends a value to an output stream. **Extraction >>:** parses input into an object. Both return the stream by reference, allowing chaining. **Manipulator:** changes formatting or performs a stream action.

```
int age; std::string name;
std::cin >> age >> name;        // Input "20 Ada Lovelace"
// age=20, name="Ada"; remaining text contains " Lovelace".
std::cout << name << ' ' << age << '\n';
```

5.2 Output formatting (<iomanip>)

Manipulator	Meaning / persistence
fixed / scientific / defaultfloat	Floating notation; persists
setprecision(n)	Default: significant digits; fixed/scientific: decimals; persists
setw(n)	Minimum field width, generally next formatted field only
setfill(ch)	Padding character; persists
left / right	Alignment in field; persists
showpos / noshowpos	Show/hide plus sign on positive numbers
showpoint / noshowpoint	Force/not force decimal point and trailing precision
boolalpha / noboolalpha	true/false text vs 1/0
dec / oct / hex	Integer base 10/8/16; persists
uppercase / nouppercase	Uppercase/lowercase exponent and hex letters
endl	Writes newline and flushes output

```
std::ostringstream out; // <sstream>, fresh settings
out << std::fixed << std::setprecision(2) << 3.14159;
// out.str() == "3.14"
std::ostringstream field;
field << std::setfill('0') << std::setw(4) << 7 << ' ' << 8;
// field.str() == "0007 8"; width resets, fill persists.
```

Width is a minimum: wider values are not truncated. setprecision changes display, not the stored value. '\n' writes a newline without explicitly flushing.

5.3 Input whitespace and C-style I/O

skipws: formatted extraction skips leading whitespace (default). **noskipws:** formatted character extraction includes whitespace. **ws:** consumes all immediately leading whitespace once; does not permanently change skipws.

```
std::istringstream in("  A B");
char a, b, c;
in >> std::noskipws >> a >> std::ws >> b >> std::skipws >> c;
// a=' ', b='A', c='B'
```

printf (<cstdio>): format-string output. %d/%i signed int; %u unsigned; %o octal; %x/%X hex; %f floating-point decimal; %e/%E scientific; %g/%G compact floating form; %a/%A hexadecimal float; %c character; %s C string; %p pointer (pass void*); %% literal percent. %n writes the count to a supplied int*, producing no text.

scanf: format-string input; pass addresses, e.g. std::scanf("%d", &age). For scanf, %f expects float*, %lf expects double*. printf("%.2f",3.14159) displays 3.14. Wrong format/argument types can cause undefined behavior.

Lecture 5 | Files, stream state and string streams

5.4 File streams and reading patterns

File: named persistent collection of data in storage. **Text file:** character data. **Opening:** associates a stream with a file. **Closing:** ends that association and releases resources. <fstream>: ifstream input; ofstream output; fstream both. Text is usually processed sequentially, unlike direct string indexing.

```
std::ifstream in("input.txt");
std::ofstream out("output.txt", std::ios::app); // Append
if (!in || !out) { /* Handle failed open */ }
std::string line;
while (std::getline(in, line)) out << line << '\n';
in.close(); out.close(); // Also closed by destructors
```

Default ofstream opening normally truncates existing contents; ios::app appends. Modern C++ accepts std::string file names as well as C strings. Closing is good resource management; opening a file does not universally lock out other programs.

getline(stream,string[,delimiter]): free function; reads through delimiter, removes it from input but does not store it. Default delimiter is newline. **get(ch):** reads next character including whitespace and returns the stream. **get():** returns a character as int, or EOF on failure/end. **put(ch):** writes a character.

```
char ch;
while (in.get(ch)) std::cout.put(ch); // Use on an open stream
// Alternative: int c; while ((c=in.get()) != EOF) ...;
// int is needed to distinguish all byte values from EOF.
```

The member in.getline(buffer,n) fills a C-string buffer; it is different from std::getline(in,line), which fills std::string.

5.5 Stream state and mixed input

eofbit: end encountered. **failbit:** extraction/format operation failed. **badbit:** serious I/O failure. **fail():** failbit or badbit; **eof():** eofbit. **Stream-to-bool conversion:** true if !fail(), enabling while(in >> x). **clear():** resets state flags, not the input buffer.

```
std::istringstream nums("12 15 x 8");
int x, sum = 0;
while (nums >> x) sum += x;   // sum=27; parsing stops at x
```

Do not loop on !in.eof(): EOF is reported after an operation encounters the end, so the last failed read may still enter the body. Test the read operation itself.

After std::cin >> number, the newline remains. To read the next whole line, discard the rest of that input line first:

```
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
std::getline(std::cin, line); // <limits> is needed
```

If parsing failed, call clear() before ignore(). getline(cin >> std::ws,line) is an alternative when leading whitespace and blank lines may be discarded intentionally.

5.6 String streams (<sstream>)

istringstream: parse a string with input operations. **ostringstream:** build a string with output operations. **stringstream:** both. **str():** gets the underlying string; str(text) replaces it. Resetting str does not clear failure flags; use clear() when reusing a failed stream.

```
std::istringstream row("Ada F 123");
std::string name; char sex; long id;
row >> name >> sex >> id;
std::ostringstream result;
result << name << '\t' << sex << '\t' << id;
std::string text = result.str(); // "Ada\tF\t123"
```

to_string(value): numeric value to std::string, e.g. "ID: " + std::to_string(123). String streams offer explicit formatting control and typed parsing.

Lecture 6 | Memory representation and pointers

6.1 Bits, bases and memory

Bit: binary digit, 0/1. **Byte:** smallest addressable C++ storage unit, typically 8 bits. **Word:** processor-dependent natural data unit; not a portable C++ type size. **RAM:** working memory. **Address:** identifies a storage location. **Address space:** set of possible addresses; N-bit byte addresses can distinguish 2^N bytes theoretically.

Base/radix: number of digits in a positional numeral system. Binary base 2; octal base 8; decimal base 10; hexadecimal base 16 (A-F = 10-15). Each digit contributes digit * base^position, starting at position 0 on the right.

```
int a = 42;      // Decimal
int b = 052;     // Octal (leading zero)
int c = 0x2A;    // Hexadecimal
int d = 0b101010;// Binary: all four equal 42
// Hex digit = 4 binary bits; octal digit = 3 bits.
```

sizeof: size in C++ bytes; produces std::size_t. sizeof(char)==1; sizes of int, pointers, enum, etc. are implementation-dependent. Arrays occupy elementCount*sizeof(element). Structs/classes can include padding; their size need not equal the sum of member sizes.

Static storage: globals and static objects, generally whole-program duration. **Automatic storage:** ordinary locals/parameters, typically stack frames. **Dynamic storage:** explicitly allocated objects, conventionally called heap objects. Physical layouts/growth directions are implementation details.

6.2 Pointer and lvalue definitions

Pointer: object/value that can identify another object/function, represent one-past an array, or be null. **Pointee:** object a pointer points to. **Address-of &:** obtains an address. **Dereference *:** accesses the pointee. **Null pointer:** points to no object; use nullptr. **Lvalue:** expression identifying an object/function; a modifiable lvalue can be assigned to. A const object is still an lvalue, but not modifiable.

```
int x = 10;
int* p = &x;
*p = 20;         // x=20
// p stores &x; &p is the separate address of pointer p.
// x and *p are modifiable lvalues; x+1 and &x are not.
```

Declaration * belongs to each declarator: int* p, q; makes p a pointer and q an int. Prefer separate declarations. int* p=nullptr; is initialized safely; dereferencing it is invalid.

6.3 Aliasing and reassignment

Aliasing: multiple names/pointers refer to the same object. Assigning p=q copies an address; assigning *p=*q copies a pointee value.

```
int x = 42, y = 163;
int* p = &y;
int* q = &x;
*p = 17;       // y=17
p = q;         // Both now point to x
*p = 9;        // x=9, y=17
```

Wild/uninitialized pointer: has no established valid target. **Dangling pointer:** refers to storage after its object lifetime ends. **Invalid dereference:** accessing through null, dangling, out-of-bounds or otherwise invalid pointers; undefined behavior.

Casting double* to int* does not convert a double to an int; reading through that incompatible pointer is generally undefined behavior. Use a numeric conversion on the value instead.

Lecture 6 | References, objects and function pointers

6.4 Pointer versus reference

Property	Pointer T*	Reference T&
Meaning	Stores a pointer value	Alias for an existing object
Initialization	Can be left uninitialized (unsafe)	Must bind when declared
Access object	*p	r
Change target	p=&other	Cannot reseat
No target	nullptr is valid pointer value	No valid null reference
Own address	&p differs from pointee address	&r is the referred object's address

Both can dangle. A reference need not have a separately observable storage slot; its implementation may use an address internally.

Pass by pointer: pointer parameter is copied, but it can modify the caller's pointee. To modify the caller's pointer itself, use T*& (reference to pointer) or T**.

```
void swapValues(int* a, int* b) {
    int t = *a; *a = *b; *b = t; // Changes pointees
}
void swapLocalPointers(int* a, int* b) {
    int* t = a; a = b; b = t;    // Only local pointer copies
}
void swapPointers(int*& a, int*& b) {
    int* t = a; a = b; b = t;    // Changes caller's pointers
}
```

For int x=1,y=2; int* p=&x; int* q=&y;, swapValues(p,q) swaps x/y; swapPointers(p,q) swaps which objects p/q point to.

6.5 Pointers to objects and this

Member selection .: selects a member on an object. **Arrow ->:** selects a member through a pointer; p->member equals (*p).member. Parentheses matter because . binds more tightly than unary *.

this: pointer to the current object inside a non-static member function. *this is the current object itself. Local/parameter names can hide members; this-> disambiguates.

```
class Point {
    int x = 0;
public:
    void setX(int x) { this->x = x; }
    int getX() const { return x; }
};
Point obj; Point* pp = &obj;
pp->setX(7);  // (*pp).setX(7); obj.getX() is now 7
```

Const member function: trailing const promises not to modify ordinary object state through this; it can be called on const objects. Static member functions have no this pointer.

6.6 Function pointers and multiple indirection

Function pointer: stores the address of a compatible function; can be passed as an argument to select behavior. **Callback:** function supplied for another function to invoke.

```
int larger(int a, int b) { return a > b ? a : b; }
int apply(int (*fn)(int,int), int x, int y) { return fn(x,y); }
// (*fn)(x,y) also calls the pointed-to function.
int answer = apply(larger, 3, 7); // 7
```

Pointer-to-pointer: points to another pointer; each dereference removes one indirection level.

```
int x = 10; int* p = &x; int** pp = &p;
**pp = 30;               // x=30
// pp==&p; *pp==p==&x; **pp==*p==x
```

Lecture 6 | Arrays and pointer arithmetic

6.7 Arrays, size and decay

Array: fixed number of contiguous objects of one element type. **Index/subscript:** zero-based position. **Allocated size/capacity:** total slots. **Effective size:** slots currently used by the program. Built-in arrays do not track effective size or check bounds.

```
int a[5] = {10,20};          // {10,20,0,0,0}
std::size_t n = sizeof a / sizeof a[0]; // 5 in array scope
int* p = a;                 // Array-to-pointer conversion
// a = p; a++;              // Invalid: array is not assignable
```

Array decay: in most expressions, an array converts to a pointer to its first element. It remains an array for sizeof a and &a. In function parameters, int a[] adjusts to int* a; pass a length separately. sizeof a there measures pointer size.

```
int sum(const int a[], int n) {
    int total = 0;
    for (int i=0; i<n; ++i) total += a[i];
    return total;
}
```

6.8 Pointer arithmetic and declarators

Declarator reading: int (*fn)(int,int) is a function pointer; int* fn(int,int) declares a function returning int*. Parentheses change the meaning.

Pointer arithmetic: p+k advances k elements, not k bytes. a[i] equals *(a+i). Subtracting pointers within the same array yields an element distance (std::ptrdiff_t). **One-past pointer:** legal endpoint for comparisons/arithmetic, never dereference it. Stay within the same array or its one-past endpoint.

```
int a[] = {10,20,30,40};
int* p = a;
*(p+2);                 // 30
(a+4) - a;              // 4
// If sizeof(int)==4 and &a[0] is 0x1000, &a[2] is 0x1008.
int* pointers[4];        // Array of four int pointers
int (*whole)[4] = &a;    // Pointer to one array of four ints
// whole+1 advances by sizeof(a), not sizeof(int).
```

Two-dimensional array: array of rows. int m[2][3] is contiguous; m decays to int (*)[3], not int**. m[i][j] equals *(*(m+i)+j). A row-pointer allocation using int** is a different representation.

6.9 Pointer increment idioms and C-string implementation

```
int a[] = {1,2,3,4};
int* p = a;
int x = *p++;   // *(p++): x=1, p points to a[1]
int y = *++p;   // *(++p): y=3, p points to a[2]
++*p;           // ++(*p): a[2]=4; p does not move
```

Postfix ++ binds before dereference. (*p)++ changes the pointee, while p++ changes the pointer.

```
std::size_t length(const char* s) {
    const char* end = s;
    while (*end != '\0') ++end;
    return static_cast<std::size_t>(end-s);
}
void copyText(char* dst, const char* src) {
    while ((*dst++ = *src++) != '\0') { }
}
```

copyText copies the terminating '\0' too; assignment yields the assigned character, so zero stops the loop. Require terminated src, sufficient dst capacity and no overlap. Outputting char* with cout normally prints C-string text; cast to const void* to display its address.

Lecture 7 | Dynamic allocation and lifetime

7.1 new, delete and ownership

Dynamic allocation: obtain storage and construct objects during execution. **new expression:** creates an object/array and returns a pointer; ordinary allocation failure throws std::bad_alloc. **delete expression:** destroys an object and frees its allocation. **Owner:** responsible for releasing a resource. **Memory leak:** allocated resource becomes unreleased/unreachable.

```
int* p = new int(5);          // Initialized scalar
int* a = new int[3]{};       // Zero-initialized array
int* raw = new int[3];       // Elements uninitialized
*p = 9;
delete p; p = nullptr;
delete[] a;
delete[] raw;
```

Match new with delete; new[] with delete[]. Do not delete automatic objects, interior array pointers, or the same allocation twice. delete nullptr is safe. Setting one pointer to nullptr does not repair other aliases.

```
int* p = new int(7);
int* q = p;
delete p;
p = nullptr;              // q is still dangling
// std::cout << *q;        // Undefined behavior
```

Lifetime: allocation lasts until the corresponding deletion, even if its original pointer variable goes out of scope. A local pointer's lifetime and its dynamic pointee's lifetime are separate. **Destructor:** special member function ~Type() called when an object is destroyed; releases owned resources. Automatic objects are destroyed at scope exit, in reverse construction order.

RAII (Resource Acquisition Is Initialization): tie resource ownership to an object's lifetime so its destructor performs cleanup. Smart pointers and file streams use this pattern. **Garbage collection:** automatic reclamation of unreachable allocations; ordinary C++ raw new allocations are not automatically garbage-collected.

7.2 Dynamic arrays and matrices

Dynamic array: contiguous array with length chosen at runtime; it does not resize itself. To change its length manually, allocate a replacement, copy elements, then delete[] the old array.

```
int* createIndexArray(int n) { // n>=0
    int* a = new int[n];
    for (int i=0; i<n; ++i) a[i]=i;
    return a;                  // Caller owns returned allocation
}
```

Row-pointer matrix: pointer array whose entries each own a row. **Flat matrix:** one contiguous allocation indexed as row*columnCount+column.

```
int rows=2, cols=3;
int** m = new int*[rows];
for (int r=0; r<rows; ++r) m[r] = new int[cols]{};
m[1][2] = 9;
for (int r=0; r<rows; ++r) delete[] m[r];
delete[] m;                   // Delete rows before pointer array
int* flat = new int[rows*cols]{};
flat[1*cols+2] = 9;
delete[] flat;
```

These short allocation examples assume allocations succeed and modest nonnegative sizes; production code needs cleanup for partial failures. A matrix class can hide storage and implement operator()(r,c) to return data[r*cols+c], preferably by reference when callers may assign.

Heap-stack diagram: draw automatic pointer p separately from its heap allocation, then follow arrows. Returning a pointer to new storage can be valid; returning &local is dangling after the function exits.

Lecture 7 | const, static and smart pointers

7.3 const and static

const object: cannot be modified through a const-qualified access path. **Pointer to const:** cannot change the pointee through that pointer. **Const pointer:** pointer cannot be reseated. **Const member:** callable on const instances and cannot ordinarily mutate their non-mutable members.

```
int x=1, y=2;
const int* p = &x;    // p=&y allowed; *p=3 forbidden
int* const q = &x;    // *q=3 allowed; q=&y forbidden
const int* const r=&x;// Neither operation allowed
```

Local static: one persistent object across calls. **Namespace/file static:** internal linkage for a function/variable. **Static data member:** shared by all instances. **Static member function:** belongs to the class, has no this, and cannot directly access instance members without an object.

```
int nextID() { static int n=0; return ++n; } // 1,2,3,...
class Counter {
public:
    static int total;
    Counter() { ++total; }
};
int Counter::total = 0; // One definition, outside the class
```

Const/reference members must be initialized in a constructor initializer list (or valid in-class initializer), not assigned for the first time inside its body.

7.4 Smart pointers (<memory>)

Smart pointer: class managing pointer ownership/lifetime. **unique_ptr:** exclusive owner, noncopyable but movable. **shared_ptr:** shared ownership; the managed object is destroyed when the last strong owner goes away. **weak_ptr:** observes a shared object without keeping it alive; lock() returns a shared_ptr if it still exists, otherwise an empty one.

```
auto u = std::make_unique<int>(7);
// auto bad = u;              // Compile error: no copying
int* observer = u.get();      // Borrowed raw pointer, no ownership
auto v = std::move(u);        // Ownership transferred; u is empty
v.reset();                   // Deletes int; observer now dangles

auto s = std::make_shared<int>(8);
std::weak_ptr<int> w = s;
{
    auto t = s;              // Two strong owners
    // s.use_count() == 2 in this isolated example
}
s.reset();                   // Last strong owner gone
if (auto live = w.lock()) { /* Not reached here */ }
// w.expired() is true.
```

Move: transfers resources from an object where the type supports it; std::move (<utility>) itself is a cast enabling move operations. **Reference count:** count of strong shared owners. A weak pointer does not increase it.

reset(): releases current ownership (or adopts a supplied compatible pointer). **get():** borrowed raw pointer, no transfer. **unique_ptr::release():** gives up ownership without deleting; caller must arrange cleanup. Avoid calling release casually.

For dynamic arrays, std::make_unique<int[]>(n) creates a unique_ptr<int[]> using delete[]. **Ownership cycle:** shared_ptr objects keeping each other alive; weak_ptr can break a non-owning back-link. Never create two independent shared_ptr owners from the same raw pointer.

Smart pointers do not validate arbitrary indexing or make borrowed raw pointers safe after destruction. Deleting through a polymorphic base also requires the appropriate virtual destructor, covered on page 20.

Lecture 8 | Classes and object design

8.1 Structures, classes and access control

Structure: compound type with named fields. **Class:** user-defined type with members and access control. In C++, struct and class have the same core capabilities; struct defaults to public members/inheritance, class to private. Both can have methods, constructors and operators.

Object/instance: particular value of a class. **Instantiation:** creating an instance. **Field/data member:** stored state. **Method/member function:** behavior. **Encapsulation:** bundle data and operations while controlling access. **Abstraction:** present useful behavior without exposing representation.

public: accessible to clients. **private:** accessible to members and friends of the class, not directly to derived-class code. **protected:** accessible within the class, friends and permitted derived-class contexts.

```
class Point {
    int x, y;                    // Private by default
public:
    Point(int x=0, int y=0) : x(x), y(y) {}
    int getX() const { return x; }
    int getY() const { return y; }
    void setX(int value) { x = value; }
    double length() const;
};
double Point::length() const {
    return std::sqrt(double(x)*x + double(y)*y);
}
Point p(3,4);              // p.length() == 5
// p.x = 9;               // Compile error: private
```

Scope-resolution :: qualifies a class/namespace member. **Getter/accessor:** reads state. **Setter/mutator:** changes state. Provide setters only if arbitrary changes preserve valid state.

8.2 Constructors, initializer lists and destruction

Constructor: member named after the class with no return type, establishes initial state. **Default constructor:** callable without arguments. **Overloaded constructors:** different parameter lists. **Initializer list:** colon-separated member/base initializations before the body. **Destructor:** ~Class(), no return type/parameters, runs at destruction.

```
class Example {
    const int id;
    int& target;
public:
    Example(int id, int& value) : id(id), target(value) {}
    ~Example() { /* Release owned resources, if any */ }
};
```

Members initialize in their declaration order, regardless of initializer-list order. Bases initialize before members. Initialization constructs a member directly; assignment in the body occurs after initialization. Reference and const members require initialization. A constructor with defaults for every parameter can serve as the default constructor. If no constructor is user-declared, the compiler declares a default constructor, which can be deleted if a member/base cannot be default-constructed.

Class invariant: condition that must hold for valid objects, maintained by public operations. **Client view:** how the class is used. **Implementer view:** fields and algorithms needed to support it. Choose a public interface, private state, constructors and operations, then implement/test them.

friend: explicitly grants an external function/class access to private/protected members. Friendship does not make the function a member. Prefer public getters when sufficient.

Lecture 8 | Copying objects and owning memory

8.3 Copy construction versus assignment

Copy constructor: initializes a new object from another: Type(const Type&). **Copy assignment:** replaces an already existing object's value: Type& operator=(const Type&). **Shallow copy:** copies pointer values, sharing pointees. **Deep copy:** duplicates owned data into separate allocations.

```
Point a(3,4);
Point b = a;      // Copy initialization (not assignment)
Point c(a);       // Copy construction
b = a;           // Copy assignment: b already exists
```

Compiler-generated copying performs memberwise copying. For an owning raw pointer, this copies only its address and can cause shared mutation and double deletion. For value members such as std::string, memberwise copying uses those members' copy behavior and can be correct.

Rule of Three: a class managing resources that needs a destructor, copy constructor or copy assignment usually needs all three. **Self-assignment:** x=x; must preserve a valid value. **this != &src:** compares object addresses. Return *this by reference from assignment to support a=b=c.

8.4 Minimal complete deep-copy example

```
class IntBuffer {
    std::size_t n;
    int* data;
public:
    explicit IntBuffer(std::size_t n)
        : n(n), data(new int[n]{}) {}

    IntBuffer(const IntBuffer& src)
        : n(src.n), data(new int[src.n]) {
        for (std::size_t i=0; i<n; ++i) data[i]=src.data[i];
    }

    IntBuffer& operator=(const IntBuffer& src) {
        if (this != &src) {
            int* fresh = new int[src.n];
            for (std::size_t i=0; i<src.n; ++i)
                fresh[i]=src.data[i];
            delete[] data;
            data = fresh;
            n = src.n;
        }
        return *this;
    }

    ~IntBuffer() { delete[] data; }
    int& operator[](std::size_t i) { return data[i]; }
    const int& operator[](std::size_t i) const { return data[i]; }
};
IntBuffer a(2); a[0]=7;
IntBuffer b=a; b[0]=9;
// a[0]==7, b[0]==9: separate owned arrays
```

The copy constructor starts without an existing owned buffer; it must not delete an uninitialized pointer. Assignment releases the destination's old data, not the source's. Allocating/copying before releasing the old buffer preserves the old value if allocation fails (int copying itself does not throw).

explicit constructor: prevents unintended implicit conversion through that constructor. **Const overload:** lets const objects read elements without allowing mutation. operator[] above assumes i<n; no bounds check is implemented.

Passing/returning objects by value can invoke copies or moves; C++ may elide copies, and some prvalue cases require elision in C++17. Do not assume every apparent temporary causes a copy. const Type& avoids an input copy when borrowing suffices.

Lecture 8 | Operators and example abstractions

8.5 Member and free operator overloads

Member binary operator: left operand is *this. **Free binary operator:** takes both operands explicitly. **Conventional value semantics:** arithmetic returns a new value; compound assignment mutates and returns the left object. Operator code can depart from that convention, so trace supplied bodies.

```
struct Vec2 {
    int x=0, y=0;
    Vec2& operator+=(const Vec2& rhs) {
        x += rhs.x; y += rhs.y; return *this;
    }
    Vec2& operator++() { ++x; ++y; return *this; }
    Vec2 operator++(int) {
        Vec2 old=*this; ++(*this); return old;
    }
};
Vec2 operator+(Vec2 a, const Vec2& b) { return a += b; }
bool operator==(const Vec2& a, const Vec2& b) {
    return a.x==b.x && a.y==b.y;
}
std::ostream& operator<<(std::ostream& os, const Vec2& v) {
    return os << '(' << v.x << ',' << v.y << ')';
}
```

Prefix: returns updated object, conventionally by reference. **Postfix:** dummy int marker, returns old value by value. **Stream overload:** takes ostream& because streams cannot be copied and returns that reference for chaining. With cout << value, the left operand is a stream, so a free function is appropriate.

Conversion function: provides conversion to another type; has no return type in its declaration. explicit operator bool() const supports conditions without broad implicit numeric conversions.

```
struct Status {
    int count;
    explicit operator bool() const { return count > 5; }
};
Status s{6};
if (s) { /* Entered: count > 5 */ }
```

Do not return a reference to a local result from operator+. It would dangle. Returning a reference to *this is valid while the original object lives.

8.6 Rational numbers, stacks and token scanners

Rational number: ratio numerator/denominator with nonzero denominator. A Rational class hides those two fields and normalizes using a GCD, keeping a positive denominator and representing zero as 0/1. Example: 2/4 normalizes to 1/2; 1/2 + 1/3 = (1*3+1*2)/(2*3) = 5/6. In general a/b - c/d = (ad-bc)/bd; multiplication gives ac/bd; division gives ad/bc (c must be nonzero). Constructors may accept zero, one or two arguments to represent 0, an integer, or a fraction.

Stack: collection accessed last in, first out (LIFO). **push:** adds at top. **pop:** removes top (the course's CharStack also returns it). **count:** current element number. **capacity:** allocated slots. **clear:** logically empties a stack, with resource behavior defined by the class. Array-backed push stores array[count++]; pop requires nonempty and reads array[--count]. A destructor frees owned storage. Standard std::stack::pop() returns void; use top() to read first.

Token: meaningful unit of text, such as a word, number or punctuation mark. **Token scanner:** object splitting input according to rules. Course operations: setInput(text/stream) sets source; hasMoreTokens() tests availability; nextToken() returns the next unit (empty string at the end); saveToken(token) puts a token back for a later read; ignoreWhitespace()/scanNumbers()/scanStrings() configure recognition. These belong to the course library, not std.

Example tokens for x = 12; might be "x", "=", "12", ";" when whitespace is ignored and numbers are grouped. Storing scanner state privately encapsulates the program's parsing progress.

Lecture 8 | Inheritance and polymorphism

8.7 Inheritance and access mapping

Inheritance: derived class reuses/extends a base class. **Base/superclass:** inherited type. **Derived/subclass:** inheriting type. Public inheritance models an is-a relationship, e.g. Circle is a Shape. **Composition:** a class contains another object (has-a), e.g. Point has coordinates.

Inheritance mode	Base public becomes	Base protected becomes
public	public	protected
protected	protected	protected
private	private	private

Base private state remains part of the base subobject but is inaccessible directly from derived code. Derived code uses accessible base operations. Inheritance mode controls access and whether clients may implicitly convert Derived* to Base*; public permits it.

8.8 Virtual functions and abstract classes

Polymorphism: one interface supports different forms. **Compile-time polymorphism:** overloading/templates choose or instantiate code at compile time. **Runtime polymorphism:** virtual dispatch chooses the overriding function for the actual dynamic object.

Virtual function: eligible for dispatch through a base pointer/reference. **Override:** matching derived function replaces virtual behavior. **Pure virtual function:** declared with =0, requires an appropriate final override for a concrete class. **Abstract class:** has an unimplemented pure virtual function and cannot be instantiated. **Concrete class:** can be instantiated.

```
class Shape {
public:
    virtual double area() const = 0;
    virtual ~Shape() = default;
};
class Circle : public Shape {
    double radius;
public:
    explicit Circle(double r) : radius(r) {}
    double area() const override {
        return 3.141592653589793 * radius * radius;
    }
};
Circle c(2);
Shape& ref = c;
double a = ref.area();       // Calls Circle::area, about 12.5664
std::unique_ptr<Shape> p = std::make_unique<Circle>(2);
// Destruction through Shape invokes Circle then Shape destructors.
```

Static type: type known from the declaration (Shape& above). **Dynamic type:** actual object's type (Circle). **Dynamic dispatch:** runtime choice using the dynamic type for virtual calls. Nonvirtual calls are selected using static type. Once a function is virtual, matching overrides remain virtual; use override to catch signature mistakes.

Virtual destructor: ensures correct derived destruction through a base pointer when using delete. A polymorphic owning base generally needs one. Destruction proceeds derived-to-base; construction proceeds base-to-derived. Base constructors can be called in the derived initializer list.

Object slicing: copying a derived object into a concrete base value loses derived-specific state/behavior. Use base references or pointers for polymorphic access. An abstract Shape cannot even be copied into a Shape value.

Overloading vs overriding: overloads have different parameter lists; overrides match a virtual base signature. A mismatched const qualifier can prevent an override. A same-named derived function can hide base overloads; overriding is not merely reusing a name.

Lecture 9 | Templates and generic algorithms

9.1 Function templates and deduction

Template: compile-time pattern for generating functions/classes. **Type parameter:** placeholder such as T. **Instantiation:** compiler generates a specialization for supplied/deduced arguments. **Deduction:** compiler infers template arguments from a call. typename and class are interchangeable when declaring an ordinary template type parameter.

```
template <typename T>
T Max(T a, T b) { return a > b ? a : b; }
// Max(1,2) -> int result 2
// Max(1.0,2.0) -> double result 2.0
// Max(std::string("cat"), std::string("dog")) -> "dog"
// Max(1,1.1);             // Deduction conflict: int vs double
// Max<double>(1,1.1);     // Explicit T=double, returns 1.1
```

Type requirement: instantiated operations must be valid for the chosen type. Max needs > and a copyable/returnable result. With ordinary const char* arguments, > does not compare text lexicographically; use std::string. A user Point requires a suitable > overload.

```
template <typename A, typename B>
A firstTypeMax(A a, B b) { return a > b ? a : b; }
// firstTypeMax(1,1.1) == 1: return type A is int.
```

The comparison may choose 1.1, but the declared return type converts it to int. Supporting mixed arguments does not itself guarantee a mixed/common return type.

9.2 Class templates and definition visibility

Class template: pattern for classes parameterized by types/values. Supply template arguments when naming a specialization, e.g. Box<int>. Template definitions generally must be visible where instantiated; the usual course approach keeps interface and implementation in the header.

```
template <typename T>
class Box {
    T value;
public:
    explicit Box(T v) : value(v) {}
    T get() const;
};
template <typename T>
T Box<T>::get() const { return value; }
Box<int> b(7);              // b.get() == 7
```

For the course's generic Stack<T>, replace char by T in the element buffer, push parameter and pop result; prefix out-of-class method definitions with template<typename T> and qualify Stack<T>::. Resource ownership/copy rules still apply.

9.3 Selection sort as a template

Selection sort: repeatedly find the smallest remaining element and swap it into the next output position. Requires comparison and swapping; its nested scan takes O(n^2) comparisons, where **O(n^2)** describes quadratic growth with input size.

```
template <typename T>
void selectionSort(T* a, int n) {
    for (int i=0; i<n; ++i) {
        int best=i;
        for (int j=i+1; j<n; ++j)
            if (a[j] < a[best]) best=j;
        std::swap(a[i], a[best]); // <utility>
    }
}
// {3,1,2} -> {1,3,2} -> {1,2,3}
```

Generic programming: write algorithms around required operations rather than a single concrete type. The same sort works for int, double and std::string when their < and swap behavior fits the intended ordering.

Lecture 9 | Iterators and range-based loops

9.4 Iterator definitions and endpoints

Container/collection: object holding elements, e.g. std::vector. **Iterator:** object representing a position and providing traversal/access operations. **Dereference *it:** accesses the element. **Increment ++it:** advances. **begin():** iterator to first element. **end():** past-the-end sentinel, never dereference. **Half-open range [begin,end):** includes first, excludes endpoint; an empty range has begin==end.

```
std::vector<int> v{10,20,30}; // <vector>
for (auto it=v.begin(); it!=v.end(); ++it)
    std::cout << *it << ' ';  // 10 20 30
```

auto: deduces a variable's type from its initializer. Explicit form: std::vector<int>::iterator it=v.begin();. **Const iterator:** prevents element mutation; v.cbegin()/v.cend() supply it. A const container's begin/end also return const iterators.

Iterator category: describes supported operations. Input: read and advance; output: write and advance; forward: multipass forward traversal; bidirectional: also --; random access: also +, -, +=, [], ordering and distance. Input iterators do not generally support --. vector iterators support random access.

```
auto it=v.begin();
it += 2;                  // Points to 30 for vector
// Do not assume += or < works for every container's iterator.
```

9.5 Range-based for: copies versus references

Range-based for: loop syntax using a range's begin/end access internally. **Value loop variable:** element copy; **reference loop variable:** aliases actual element.

```
for (auto x : v) x += 1;          // v stays {10,20,30}
for (auto& x : v) x += 1;         // v becomes {11,21,31}
for (const auto& x : v)           // Read without element copies
    std::cout << x << ' ';
```

Range-for requires an iterable range; standard std::stack and std::queue do not expose begin/end. A course custom Stack may implement iteration. Likewise std::map iteration yields key/value pairs, whereas a course custom map may yield keys.

9.6 Reverse traversal and implementation

Reverse iterator: traverses the opposite direction; rbegin() starts at the last element, rend() marks past the reverse end. ++ on a reverse iterator moves toward earlier forward elements.

```
for (auto it=v.rbegin(); it!=v.rend(); ++it)
    std::cout << *it << ' ';      // 31 21 11
// With bidirectional forward iterators, including empty v:
for (auto it=v.end(); it!=v.begin(); )
    std::cout << *--it << ' ';    // Decrement before dereference
```

Nested class: class declared inside another; can name/access enclosing private members, but needs an enclosing object to access its non-static data. A simple custom vector iterator stores a container pointer and index; * accesses that index, ++ increments it, and equality compares container identity and index. begin uses index 0; end uses element count. Member operator++() and operator++(int) implement prefix/postfix forms.

Iterator invalidation: mutations may make iterators unusable. Vector reallocation invalidates all iterators/references/pointers to its elements; other mutations have operation-specific rules. Avoid changing container size during traversal unless using a documented safe pattern.

Final review | Tracing checklist and source map

Fast exam checklist

When you see...	Ask / remember
a/b	Are both operands integers? Cast before dividing.
a++ versus ++a	Which value does the expression yield?
if (x=...)	Assignment or intended comparison?
AND / OR	Is the right operand skipped?
function call	Value copy, reference alias, or copied pointer?
p=q versus *p=*q	Copy address or copy pointee value?
&p versus p	Address of pointer or pointer's stored target?
*p++	Read old pointee, then advance pointer.
a[i]	Is i within the element range?
sizeof a	Actual array or adjusted pointer parameter?
char buffer	Space for '\0'? Termination guaranteed?
string +	At least one actual std::string operand?
find	Check against std::string::npos.
input loop	Test the read, not !eof().
setprecision	Significant digits or decimal places?
new/new[]	Matching delete/delete[] and a clear owner?
copy of class	Do pointer fields own shared or independent data?
constructor	Member declaration order; bases first.
virtual call	Static type, dynamic type and matching override?
template	Deduced types, required operations and return conversion?
iterator	Is it dereferenceable? Is end excluded? Is it still valid?

Sources | Lecture slide map

Original lecture slide decks

All page numbers below are PDF page numbers. The guide follows these original slides; it does not use the generated mock midterm as evidence of exam scope. Some examples expand or correct slide code to state precise C++17 behavior.

L1: 1-CSC3002_Lecture Notes Overview of C++.pdf, pp. 14-64. Program structure, compilation, namespaces, types, scope, expressions, control flow, assertions. Introductory swap/template previews consolidate into L2/L9.

L2: 2 - CSC3002_Lecture Notes Functions and Libraries Part 1.pdf, pp. 3-35. Functions, algorithms, GCD, overloads, defaults, calls, references, operator examples.

L3: 3-CSC3002_Lecture Notes Functions and Libraries Part 2.pdf, pp. 4-34. Libraries, math, interface/implementation, enum Direction, exported constants, design principles and random-number library.

L4: 4-CSC3002_Lecture Notes_Strings.pdf, pp. 4-38. Characters, cctype/cstring, C++ string operations and applications.

L5: 5-CSC3002_Lecture Notes Streams.pdf, pp. 4-38. C-style I/O, streams/manipulators, files, character/line reads, conversion to bool and string streams.

L6: 6-CSC3002_Pointers and Arrays(2).pdf, pp. 3-46. Bases, memory, pointers, lvalues, references, object/function pointers, arrays, pointer arithmetic and C-string implementations.

L7: 7-CSC3002 Dynamic memory management.pdf, pp. 3-30. Allocation, arrays/matrices, cleanup, destructors, diagrams, const/static and smart pointers. C allocation slide is explicitly marked awareness-only; malloc allocates raw storage, free releases it, and they must not be mixed with new/delete.

L8: 8-CSC3002_Designing Classes(1).pdf, pp. 3-57. Enums, structures/classes, getters/setters, construction/copying, operators, stacks, Rational, token scanners, inheritance and polymorphism.

L9: 9-CSC3002_Template and Iterator.pdf, pp. 3-22. Overloads, templates, type requirements, generic sorting, class templates, iterator implementation and traversal.